

# Tree

## 平衡树

### \_\_gnu\_pbds::tree

附：[官方文档地址](#)

```
1 #include <LibText/pb_ds/assoc_container.hpp> // 因为 tree 定义在这里 所以需要包含这个头文件
2 #include <LibText/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4 __gnu_pbds::tree<Key, Mapped, Cmp_Fn = std::less<Key>, Tag = rb_tree_tag,
5                 Node_Update = null_tree_node_update,
6                 Allocator = std::allocator<char>>
```

## 模板形参

- Key: 储存的元素类型，如果想要存储多个相同的 Key 元素，则需要使用类似于 std::pair 和 struct 的方法，并配合使用 lower\_bound 和 upper\_bound 成员函数进行查找
- Mapped: 映射规则（Mapped-Policy）类型，如果要指示关联容器是 集合，类似于存储元素在 std::set 中，此处填入 null\_type，低版本 g++ 此处为 null\_mapped\_type；如果要指示关联容器是 带值的集合，类似于存储元素在 std::map 中，此处填入类似于 std::map<LTKey, Value> 的 Value 类型
- Cmp\_Fn: 关键字比较函子，例如 std::less<LTKey>
- Tag: 选择使用何种底层数据结构类型，默认是 rb\_tree\_tag。\_\_gnu\_pbds 提供不同的三种平衡树，分别是：
  - rb\_tree\_tag: 红黑树，一般使用这个，后两者的性能一般不如红黑树
  - splay\_tree\_tag: splay 树
  - ov\_tree\_tag: 有序向量树，只是一个由 vector 实现的有序结构，类似于排序的 vector 来实现平衡树，性能取决于数据想不想卡你
- Node\_Update: 用于更新节点的策略，默认使用 null\_node\_update，若要使用 order\_of\_key 和 find\_by\_order 方法，需要使用 tree\_order\_statistics\_node\_update
- Allocator: 空间分配器类型

## 构造方式

```
1 __gnu_pbds::tree<std::pair<int, int>, __gnu_pbds::null_type,
2                 std::less<std::pair<int, int>>, __gnu_pbds::rb_tree_tag,
3                 __gnu_pbds::tree_order_statistics_node_update>
4 trr;
```

## 成员函数

- insert(x): 向树中插入一个元素 x，返回 std::pair<LTpoint\_iterator, bool>，其中第一个元素代表插入位置的迭代器，第二个元素代表是否插入成功。
- erase(x): 从树中删除一个元素/迭代器 x。如果 x 是迭代器，则返回指向 x 下一个的迭代器（如果 x 是 end() 则返回 end()）；如果 x 是 Key，则返回是否删除成功（如果不存在则删除失败）。
- order\_of\_key(x): 返回严格小于 x 的元素个数（以 Cmp\_Fn 作为比较逻辑），即从 开始的排名。
- find\_by\_order(x): 返回 Cmp\_Fn 比较的排名所对应元素的迭代器。
- lower\_bound(x): 返回第一个不小于 x 的元素所对应的迭代器（以 Cmp\_Fn 作为比较逻辑）。

- `upper_bound(x)`: 返回第一个严格大于 `x` 的元素所对应的迭代器（以 `Cmp_Fn` 作为比较逻辑）。
- `join(x)`: 将 `x` 树并入当前树, `x` 树被清空（必须确保两树的 **比较函数** 和 **元素类型** 相同）。
- `split(x,b)`: 以 `Cmp_Fn` 比较, 小于等于 `x` 的属于当前树, 其余的属于 `b` 树。
- `empty()`: 返回是否为空。
- `size()`: 返回大小。

#### ▼ 注意

`join(x)` 函数需要保证并入树的键的值域与被并入树的键的值域 **不相交**（也就是说并入树内所有值必须全部大于/小于当前树内的所有值），否则会抛出 `join_error` 异常。

如果要合并两棵值域有交集的树，需要将一棵树的元素一一插入到另一棵树中。

## 示例

```

1 // Common Header Simple over C++11
2 #include <LTIostream>
3 using namespace std;
4 using ll = long long;
5 using ull = unsigned long long;
6 using ld = long double;
7 using pii = pair<int, int>;
8 #include <LText/pb_ds/assoc_container.hpp>
9 #include <LText/pb_ds/tree_policy.hpp>
10 __gnu_pbds::tree<pair<int, int>, __gnu_pbds::null_type, less<pair<int, int>>,
11                 __gnu_pbds::rb_tree_tag,
12                 __gnu_pbds::tree_order_statistics_node_update>
13     trr;
14
15 int main() {
16     int cnt = 0;
17     trr.insert(make_pair(1, cnt++));
18     trr.insert(make_pair(5, cnt++));
19     trr.insert(make_pair(4, cnt++));
20     trr.insert(make_pair(3, cnt++));
21     trr.insert(make_pair(2, cnt++));
22     // 树上元素 {(1,0), (2,4), (3,3), (4,2), (5,1)}
23
24     auto it = trr.lower_bound(make_pair(2, 0));
25     trr.erase(it);
26     // 树上元素 {(1,0), (3,3), (4,2), (5,1)}
27
28     // 输出排名 0 1 2 3 中的排名 1 的元素的 first
29     auto it2 = trr.find_by_order(1);
30     cout << (*it2).first << endl; // 输出: 3
31
32     // 输出其排名
33     int pos = trr.order_of_key(*it2);
34     cout << pos << endl; // 输出: 1
35
36     // 按照 it2 分裂 trr
37     decltype(trr) newtr;
38     trr.split(*it2, newtr);
39     for (auto i = newtr.begin(); i != newtr.end(); ++i) {
40         cout << (*i).first << ' '; // 输出: 4 5
41     }
42     cout << endl;
43
44     // 将 newtr 树并入 trr 树, newtr 树被清空。
45     trr.join(newtr);
46     for (auto i = trr.begin(); i != trr.end(); ++i) {
47         cout << (*i).first << ' '; // 输出: 1 3 4 5
48     }
49     cout << endl;
50     cout << newtr.size() << endl; // 输出: 0
51
52     return 0;
53 }

```

## 参考资料

- [Tree-Based Containers](#)
  - [join 函数在 GCC 14.1.0 中的实现](#)
  - [erase 函数在 GCC 14.1.0 中的实现](#)
-

本页面最近更新：2024/11/20 19:30:43, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [ChungZH](#), [HeRaNO](#), [Ir1d](#), [isdanni](#), [ksyx](#), [lleixx](#), [opsiff](#), [StableAgOH](#), [Tiphereth-A](#), [Xeonacid](#), [zyzyh](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用